

GuardGap end-to-end showcase

Run date: September 28, 2026
Application: Support Agent Showcase (`guardgap-showcase`)
Environment: `staging`
Repository identity: `acme/support-agent`

Executive result

GuardGap successfully processed two complete synthetic deployment snapshots through the same collector, scoped-token API, database, assessment engine, and dashboard used for a real integration.

Snapshot	Met	Gaps	Unknown	Outcome
Before remediation	1	7	1	Broad IAM, public invocation, public and unencrypted SQL, object ACLs, and a failed cross-customer authorization test were detected.
After remediation	9	0	0	All nine supported controls were backed by current matching evidence.

The second assessment recorded eight transitions: seven `gap → met` changes and one `unknown → met` change. The immutable build check remained met in both snapshots.

WORKSPACE

Overview

Architecture

Control library

Connections

Activity

Self-hosted workspace

GuardGap 0.1.0

Sign out

APPLICATION POSTURE

See the gaps. Track the progress.

Security controls grounded in the evidence your pipeline and cloud actually provide.

Import evidence Reassess snapshot

Environment staging Commit bbbbbbbbbb Collected Sep 28, 12:02 AM Repository acme/support-agent

Checks assessed
9
Across 6 observed resources

Controls met
9
Supported by current evidence

Control gaps
0
Actionable configuration findings

Need evidence
0
Incomplete, stale, or planned only

Control assessment			
Select a check to inspect evidence and remediation.			
All checks Gaps Need evidence Met JSON			
CONTROL / RESOURCE	PRIORITY	EVIDENCE	STATUS
Selected runtime identities have no basic broad roles IAM-001 · showcase-project IAM	high	deployed gcp_api	met
Backend invocation requires IAM authentication RUN-001 · support-agent-api	high	deployed gcp_api	met
Service uses a dedicated runtime identity RUN-002 · support-agent-api	medium	deployed gcp_api	met
Ready revision matches the collected build digest RUN-003 · support-agent-api	high	deployed gcp_api	met
Database has no public IP SQL-001 · customer-records	high	deployed gcp_api	met
Database requires encrypted connections SQL-002 · customer-records	medium	deployed gcp_api	met
Bucket explicitly enforces public access prevention GCS-001 · support-documents	high	deployed gcp_api	met
Bucket uses uniform access control GCS-002 · support-documents	medium	deployed gcp_api	met
Negative authorization test passed for this image TEST-001 · test_cross_customer_denied	high	test junit	met

What changed

Compared with the preceding snapshot for this repository and environment.

IAM-001 showcase-project IAM	gap → met
RUN-001 support-agent-api	gap → met
RUN-002 support-agent-api	gap → met
SQL-001 customer-records	gap → met
SQL-002 customer-records	gap → met
GCS-001 support-documents	unknown → met
GCS-002 support-documents	gap → met
TEST-001 test_cross_customer_denied	gap → met

Not observed means missing from this snapshot, not resolved.

Assessment history

All environments

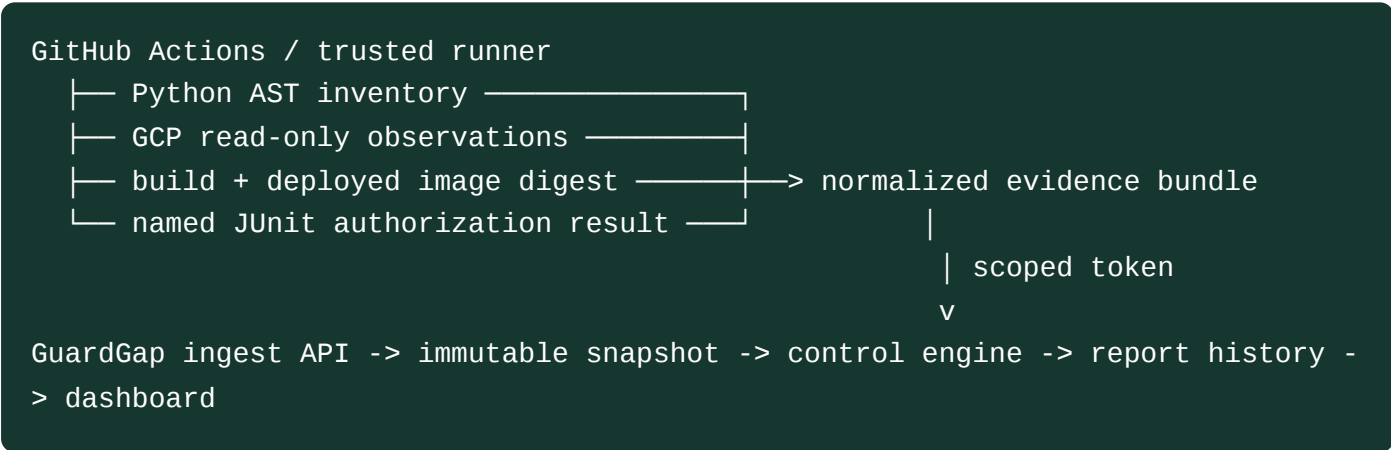
Sep 28, 12:02 AM staging · bbbbbbbb	0 gaps / 9 met
Sep 28, 12:02 AM staging · aaaaaaaa	7 gaps / 1 met

Checks apply to selected resources under a private-backend baseline. MET confirms the stated check within collected evidence, not complete security. Collector identity is trusted; observations are not independently attested.

What was exercised

The showcase uses the runnable support-agent fixture in `examples/support-agent` and the packaged GitHub Actions collector hook in `examples/github/collect.yml`. No real cloud account was changed. The GCP API responses are synthetic, while source scanning, JUnit parsing, token authentication, HTTP ingestion, persistence, comparison, rendering, and audit logging are executed by the live GuardGap service.

1. A `guardgap-showcase` workspace was created through the authenticated application API.
2. An environment-scoped `staging` collector token was created through the token API.
3. The collector scanned the real Python fixture without executing its source.
4. It normalized synthetic Cloud Run, Cloud SQL, Cloud Storage, and project IAM observations.
5. It parsed the named `test_cross_customer_denied` JUnit result and tied it to the same image digest as the ready Cloud Run revision.
6. It submitted the bundle to `POST /api/ingest` with the scoped bearer token.
7. GuardGap persisted the immutable snapshot, ran nine deterministic checks, and created the first report.
8. A second build/deployment snapshot was submitted after the dummy remediations. GuardGap compared it with the first compatible snapshot and recorded the status transitions.



Observed architecture

The final bundle contains six resources and four evidence-backed connections:

From	To	Relationship	Evidence basis
<code>support-agent</code> source	<code>support-agent-api</code>	Build digest match	Collector-supplied build digest matches the ready revision digest.
<code>test_cross_customer_denied</code>	<code>support-agent-api</code>	Test image match	JUnit test digest matches the ready deployed image.
<code>support-agent-api</code>	<code>customer-records</code>	Cloud SQL attachment	Exact Cloud SQL connection name matches the ready revision annotation.
<code>support-agent-api</code>	<code>showcase-project IAM</code>	Runtime identity policies	Ready revision service account is correlated with collected IAM policies.

The Storage bucket remains visible in inventory without an invented application data-flow edge. This is intentional: GuardGap only draws relationships supported by collected configuration.

GuardGap

WORKSPACE

Overview

Architecture

Control library

Connections

Activity

Self-hosted workspace

GuardGap 0.1.0

Sign out

Workspace / Architecture

Support Agent Showcase + Application

APPLICATION MAP

From code to cloud.

Every connection has a source. Unconnected resources remain visible without invented data flows.

staging - acme/support-agent - commit bbbbbb6 resources - 4 connections

Observed architecture

Click a component to inspect its facts

support-agent

PYTHON COMPONENT - SOURCE

test_cross_customer_de...

AUTHORIZATION TEST - TEST

support-agent-api

CLOUD RUN - DEPLOYED

customer-records

CLOUD SQL - DEPLOYED

support-documents

GCS BUCKET - DEPLOYED

showcase-project IAM

PROJECT IAM - DEPLOYED

Connection evidence

FROM -> TO	RELATIONSHIP	SUPPORTING EVIDENCE
support-agent-api -> showcase-project IAM	runtime identity policies evidenced	support-agent@showcase-project.iam.gserviceaccount.com
support-agent-api -> customer-records	Cloud SQL attachment evidenced	Exact connectionName matches the ready revision's Cloud SQL annotation; traffic is not observed.
test_cross_customer_denied -> support-agent-api	test image match evidenced	JUnit supplied with the same image digest; the CI producer is trusted.
support-agent -> support-agent-api	build digest match evidenced	Runner supplies this checkout's build digest; deployed revision matches. Independent provenance verification is not performed.

Resource inventory

COMPONENT	TYPE	LAYER	SOURCE
support-agent	python_component	source	python_ast
showcase-project IAM	project_iam	deployed	gcp_api
support-agent-api	cloud_run	deployed	gcp_api
customer-records	cloud_sql	deployed	gcp_api
support-documents	gcs_bucket	deployed	gcp_api
test_cross_customer_denied	authorization_test	test	junit

GuardGap

Evidence-backed checks. Explicit uncertainty.

Before remediation

The first deployment deliberately contained:

- A default runtime identity with a direct Editor grant.
- Disabled Cloud Run Invoker IAM and a public invoker binding.
- A matching immutable deployed image, which correctly remained met.
- A public Cloud SQL address that allowed unencrypted connections.
- Inherited rather than explicit Storage public-access prevention.
- Object ACL support through disabled uniform bucket-level access.
- A failed cross-customer negative authorization test.

WORKSPACE

Overview

Architecture

Control library

Connections

Activity

Self-hosted workspace

GuardGap 0.1.0

Sign out

APPLICATION POSTURE

See the gaps. Track the progress.

Security controls grounded in the evidence your pipeline and cloud actually provide.

Import evidence

Reassess snapshot

Environment staging Commit aaaaaaaaaa Collected Sep 28, 12:02 AM Repository acme/support-agent

Checks assessed

9

Across 6 observed resources

Controls met

1

Supported by current evidence

Control gaps

7

Actionable configuration findings

Need evidence

1

Incomplete, stale, or planned only

Control assessment

Select a check to inspect evidence and remediation.

All checks

Gaps

Need evidence

Met

JSON

CONTROL / RESOURCE

PRIORITY

EVIDENCE

STATUS

Selected runtime identities have no basic broad roles

IAM-001 - showcase-project IAM

high

deployed

gcp_api

gap

Backend invocation requires IAM authentication

RUN-001 - support-agent-api

high

deployed

gcp_api

gap

Service uses a dedicated runtime identity

RUN-002 - support-agent-api

medium

deployed

gcp_api

gap

Ready revision matches the collected build digest

RUN-003 - support-agent-api

high

deployed

gcp_api

met

Database has no public IP

SQL-001 - customer-records

high

deployed

gcp_api

gap

Database requires encrypted connections

SQL-002 - customer-records

medium

deployed

gcp_api

gap

Bucket explicitly enforces public access prevention

GCS-001 - support-documents

high

deployed

gcp_api

unknown

Bucket uses uniform access control

GCS-002 - support-documents

medium

deployed

gcp_api

gap

Negative authorization test passed for this image

TEST-001 - test_cross_customer_denied

high

test

junit

gap

What changed

Compared with the preceding snapshot for this repository and environment.

IAM-001
showcase-project IAM

new → gap

RUN-001
support-agent-api

new → gap

RUN-002
support-agent-api

new → gap

RUN-003
support-agent-api

new → met

SQL-001
customer-records

new → gap

SQL-002
customer-records

new → gap

GCS-001
support-documents

new → unknown

GCS-002
support-documents

new → gap

TEST-001
test_cross_customer_denied

new → gap

Not observed means missing from this snapshot, not resolved.

Assessment history

All environments

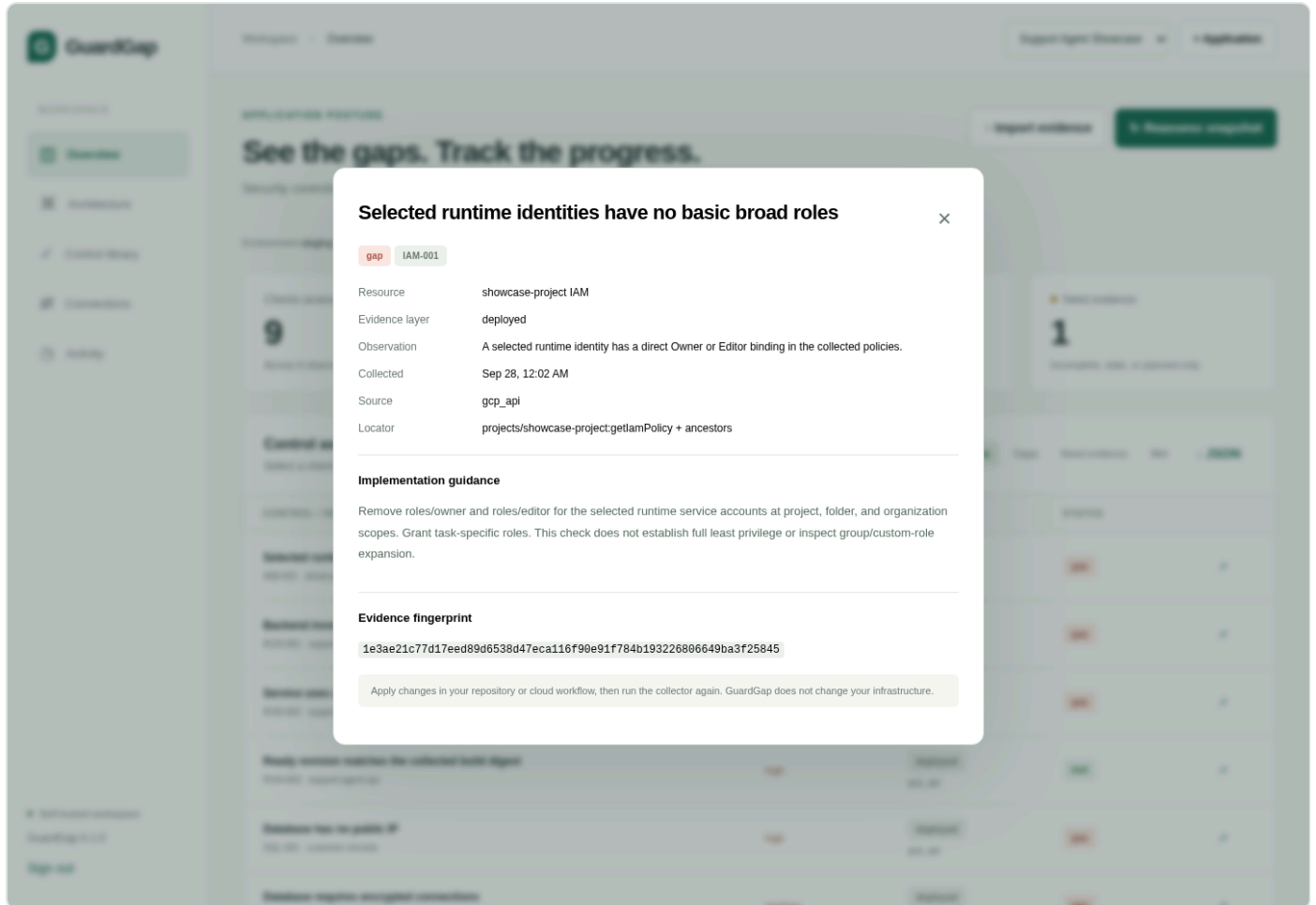
Sep 28, 12:02 AM
staging - bbbbbbbb

0 gaps / 9 met

Sep 28, 12:02 AM
staging - aaaaaaaa

7 gaps / 1 met

Each finding exposes the normalized observation, evidence source, locator, timestamp, fingerprint, and concrete implementation guidance.



After remediation

The second snapshot represented these deployed changes:

Control	Before	After	Evidence change
IAM-001	Gap	Met	Runtime account no longer has direct Owner or Editor bindings.
RUN-001	Gap	Met	Invoker IAM is enabled and no public invoker binding is present.
RUN-002	Gap	Met	Ready revision uses a dedicated service account.
RUN-003	Met	Met	Ready revision still matches the collector-supplied immutable build digest.
SQL-001	Gap	Met	Public IPv4 is disabled and no public address is present.
SQL-002	Gap	Met	SSL mode is <code>ENCRYPTED_ONLY</code> .
GCS-001	Unknown	Met	Public access prevention is explicitly <code>enforced</code> .

Control	Before	After	Evidence change
GCS-002	Gap	Met	Uniform bucket-level access is enabled.
TEST-001	Gap	Met	The named cross-customer denial test passes against the deployed image digest.

The dashboard displays both reports in history and shows each transition in the “What changed” panel.

Collector hook and token lifecycle

The Connections view produces the command for the selected application and documents the GitHub Actions + GCP integration boundary. Collector credentials are scoped to one application and environment. The unused setup token from this run was revoked; one active showcase token remains, demonstrating that revocation is retained independently of assessment history.

GuardGap

Workspace

Connections

Support Agent Showcase

+ Application

WORKSPACE

Overview

Architecture

Control library

Connections

Activity

Self-hosted workspace

GuardGap 0.1.0

Sign out

CONNECTIONS

Run anywhere. Collect where it lives.

Your collectors connect outward to this service. Cloud credentials stay on the runner.

Connect your pipeline

GitHub Actions + GCP

1 Select an application

Current application: guardgap-showcase

2 Create an environment-scoped token

Store the token as GUARDGAP_TOKEN in GitHub environment secrets. It can submit evidence only for this application and environment.

+ Collector token

3 Set the destination

Set GUARDGAP_URL to your reachable HTTPS address. A GitHub-hosted runner cannot reach localhost on your laptop; use a self-hosted runner on your network for local hosting.

http://localhost:8000

4 Collect after deployment

Use the packaged workflow in examples/github/collect.yml. Authenticate the runner to GCP with Workload Identity Federation, then run:

python -m guardgap collect --application guardgap-showcase --environment staging --repo . --repository OWNER/REPO --gcp-project YOUR_PROJECT --region us-central1 --run-service YOUR_BACKEND --sql-instance YOUR_DATABASE --bucket YOUR_BUCKET --image-digest sha256:BUILD_DIGEST --deployment-status succeeded --url "\$GUARDGAP_URL"

Remove flags for resources you do not use. Add --terraform-plan for a saved plan and --junit for explicit test evidence. The collector uses your Git checkout or GITHUB_SHA.

Collector access

Showcase CI hook

staging - Sep 28, 12:01 AM

revoked

Showcase CI hook

staging - Sep 28, 12:02 AM

Revoke

Offline collection

Omit --url to write a normalized JSON bundle. Transfer it securely, then import it here. The source timestamps remain intact.

+ Import evidence

Integration boundary

Included: Python source inventory, saved Terraform plans, GCP Cloud Run / Cloud SQL / Storage / IAM, and named JUnit evidence.

Other cloud providers and arbitrary languages require adapters. Architecture edges are created only when the collector can support a relationship.

GuardGap

Evidence-backed checks. Explicit uncertainty.

The production-ready hook template is [examples/github/collect.yml](#). It authenticates to GCP through Workload Identity Federation, runs the collector after deployment, generates a Markdown assessment, and uploads the normalized evidence as a workflow artifact.

Audit evidence

The live audit trail records application creation, token creation and revocation, and both evidence-ingestion events. Snapshot and token identifiers are recorded without exposing the bearer-token value.

GuardGap

WORKSPACE

Overview

Architecture

Control library

Connections

Activity

Workspace / Activity

Support Agent Showcase + Application

AUDIT TRAIL

A record of every assessment.

Evidence ingestion, reassessment, application setup, and collector access changes.

Sep 28, 12:08 AM

token.revoked
token_id: e00bec84-a55a-46b3-a1e4-e05fc6dd0cd5

Sep 28, 12:02 AM

evidence.ingested
application_id: guardgap-showcase · environment: staging · snapshot_id: c2c2c5c6-4bb2-4f66-8013-8ffa9f89ba2 · token_id: f9a9530-aa9a-47bd-aec8-5a6fc2b022fb · demo: false

Sep 28, 12:02 AM

evidence.ingested
application_id: guardgap-showcase · environment: staging · snapshot_id: f543bd08-9f47-4f55-9880-0811d81e7470 · token_id: f9a9530-aa9a-47bd-aec8-5a6fc2b022fb · demo: false

Sep 28, 12:02 AM

token.created
application_id: guardgap-showcase · environment: staging

Sep 28, 12:01 AM

token.created
application_id: guardgap-showcase · environment: staging

Sep 28, 12:01 AM

application.created
application_id: guardgap-showcase

Evidence package

Artifact	Purpose
evidence-before.json	Normalized collector bundle submitted for the vulnerable deployment.
evidence-after.json	Normalized collector bundle submitted for the remediated deployment.
gcp-before.json / gcp-after.json	Synthetic read-only GCP API observations consumed by the real normalizer.
authorization-before.xml / authorization-after.xml	Failed and passing named JUnit authorization evidence.
assessment-before.md / assessment-after.md	Standalone CLI reports generated from each evidence bundle.
showcase-summary.json	Report IDs, snapshot IDs, counts, transitions, and architecture totals.
SHA256SUMS.txt	SHA-256 fingerprints for the source evidence artifacts.

Validation

- Production image build: passed on Python 3.12.
- Container health check: healthy; `/healthz` returned GuardGap `0.1.0`.
- Automated suite: **16 passed**.
- CLI-to-live-service test: passed, including application/token creation, two collector uploads, report history, and `7 gaps / 1 met / 1 unknown → 9 met` transition.
- Live showcase ingestion: passed with two persisted snapshots, two reports, six resources, four architecture connections, and eight recorded status transitions.
- Authenticated screenshot capture: passed against the running service at `http://localhost:8000`.

Scope statement

This package demonstrates GuardGap's software flow and supported control logic. The GCP observations are intentionally synthetic and do not prove that a real GCP project has these settings. A production showcase should run the same packaged collector with a read-only GCP identity after an actual deployment. A `MET` result confirms only the named predicate within collected evidence; it is not a complete security certification.